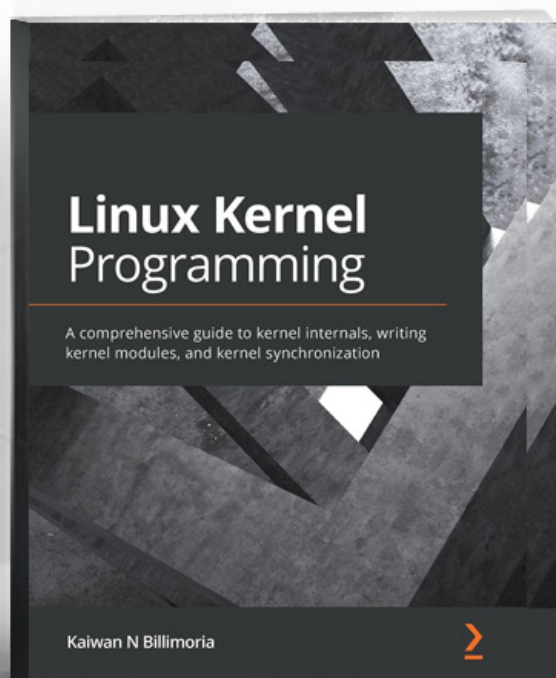


Linux Kernel Programming

‘Linux Kernel Programming’ is targeted at people who are quite new to the world of Linux kernel development, and makes no assumptions regarding their knowledge of the kernel. The prerequisite to reading this book is only a working knowledge of programming on Linux with ‘C’, as that is the medium that is used throughout it (along with a few bash scripts).



This comprehensive guide to kernel internals, writing kernel modules, and kernel synchronisation was published by Packt in March 2021. All the material and code examples in the book are based on the 5.4 LTS Linux kernel. This kernel is slated to be maintained right through December 2025, thus keeping the book’s content very relevant for a long while! It can be a handy resource for people dealing with device drivers, embedded Linux, the Linux kernel or Linux programming.

The book is divided into three major sections. The extract that follows is taken from Chapter 12 of the book. Titled ‘Kernel Synchronization - Part 1’, this chapter falls under section 3 ‘Delving Deeper’.

Part 1: Kernel Synchronization Chapter 12: Concepts - the lock

We require synchronization because of the fact that, without any intervention, threads can concurrently execute critical sections where shared writeable data (shared state) is being worked upon. To defeat concurrency, we need to get rid of

parallelism, and we need to *serialize* code that’s within the critical section – the place where the shared data is being worked upon (for reading and/or writing).

To force a code path to become serialized, a common technique is to use a lock. Essentially, a lock works by guaranteeing that precisely one thread of execution can “take” or own the lock at any given point in time. Thus, using a lock to protect a critical section in your code will give you what we’re after – running the critical section’s code exclusively (and perhaps atomically; more on this to come).

Figure 12.3 shows one way to fix the situation mentioned previously: using a lock to protect the critical section! How does the lock (and unlock) work, conceptually?

The basic premise of a lock is that whenever there is contention for it – that is, when multiple competing threads (say, *n* threads) attempt to acquire the lock (the LOCK operation) – exactly one thread will succeed. This is called the “winner” or the “owner” of the lock. It sees the *lock* API as a non-blocking call and thus continues to run happily – and exclusively – while executing the code of the critical section